

DEBCOVDIFF: Differential Testing of Coverage Measurement Tools on Real-World Projects

Wentao Zhang, Jinghao Jia, Erkai Yu, Darko Marinov, Tianyin Xu



ASE 2025

Seoul, South Korea

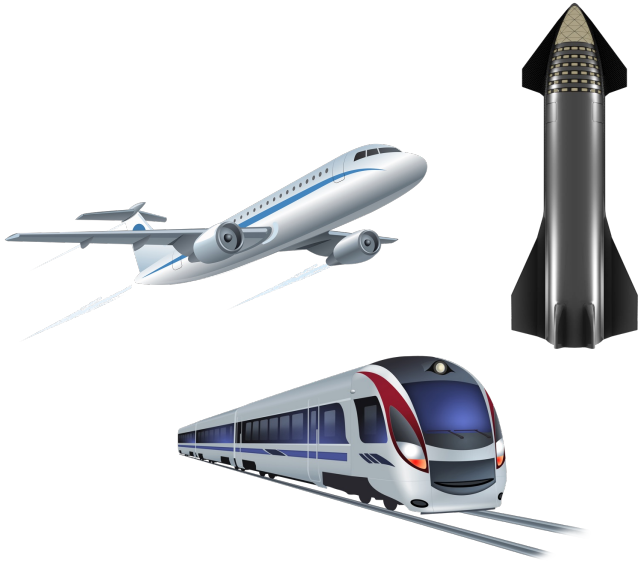
November 17, 2025



CNS-1956007
CCF-1956374
CNS-2145295

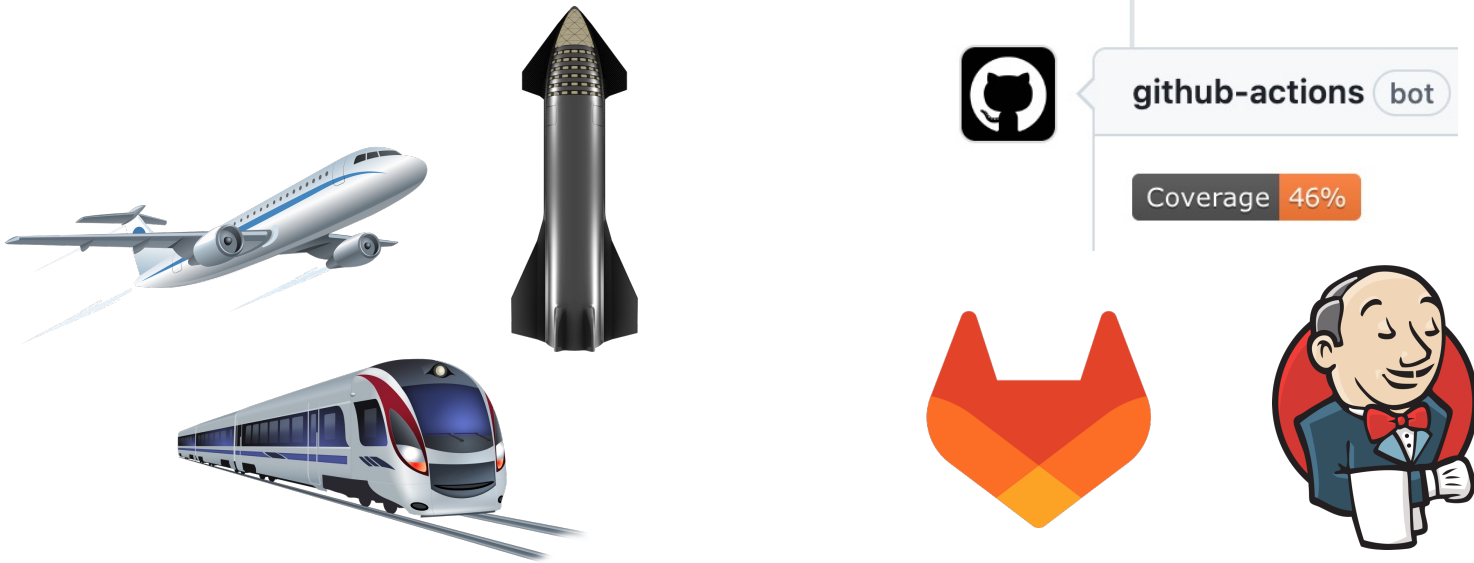


Why Code Coverage Matters



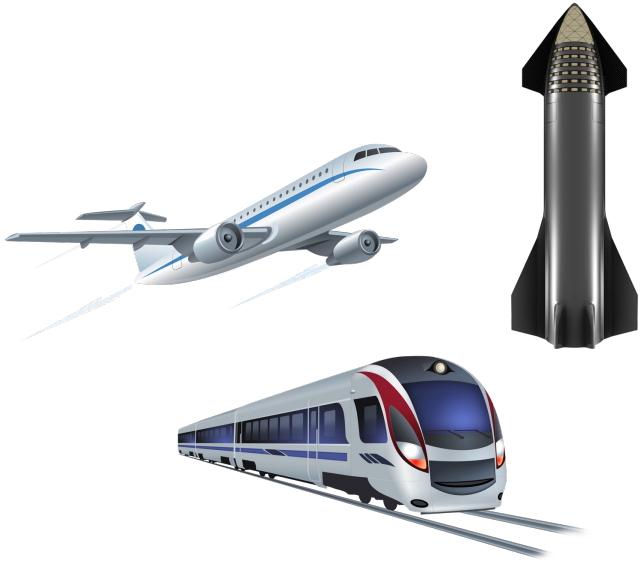
- **Certification** in safety-critical industries
 - Aviation (DO-178C)
 - Automotive (ISO-26262)

Why Code Coverage Matters

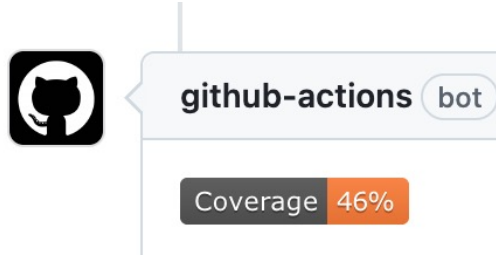


- **Certification** in safety-critical industries
 - Aviation (DO-178C)
 - Automotive (ISO-26262)
- Everyday **development**
 - Continuous integration (GitHub Actions)
 - Tech companies such as Google [Ivanković et al. 2019]

Why Code Coverage Matters



- **Certification** in safety-critical industries
 - Aviation (DO-178C)
 - Automotive (ISO-26262)

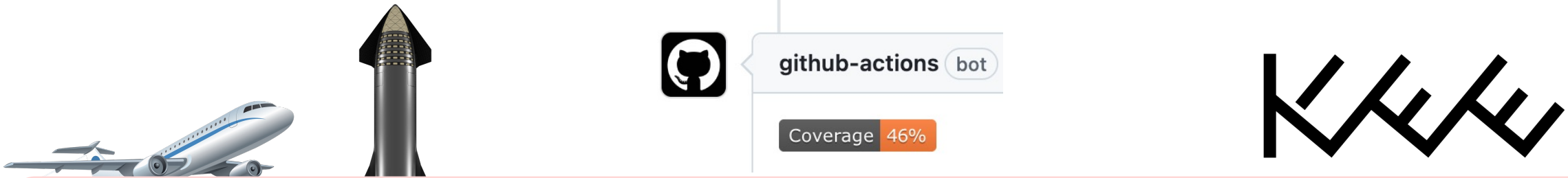


- Everyday **development**
 - Continuous integration (GitHub Actions)
 - Tech companies such as Google [Ivanković et al. 2019]



- Advanced **program analysis**
 - Fuzzing (AFL, libFuzzer)
 - Fault localization [Jones et al. 2005]

Why Code Coverage Matters



...only when the reported coverage is *correct*!

critical industries

- Aviation (DO-178C)
- Automotive (ISO-26262)

- Continuous integration (GitHub Actions)
- Tech companies such as Google [Ivanković et al. 2019]

analysis

- Fuzzing (AFL, libFuzzer)
- Fault localization [Jones et al. 2005]

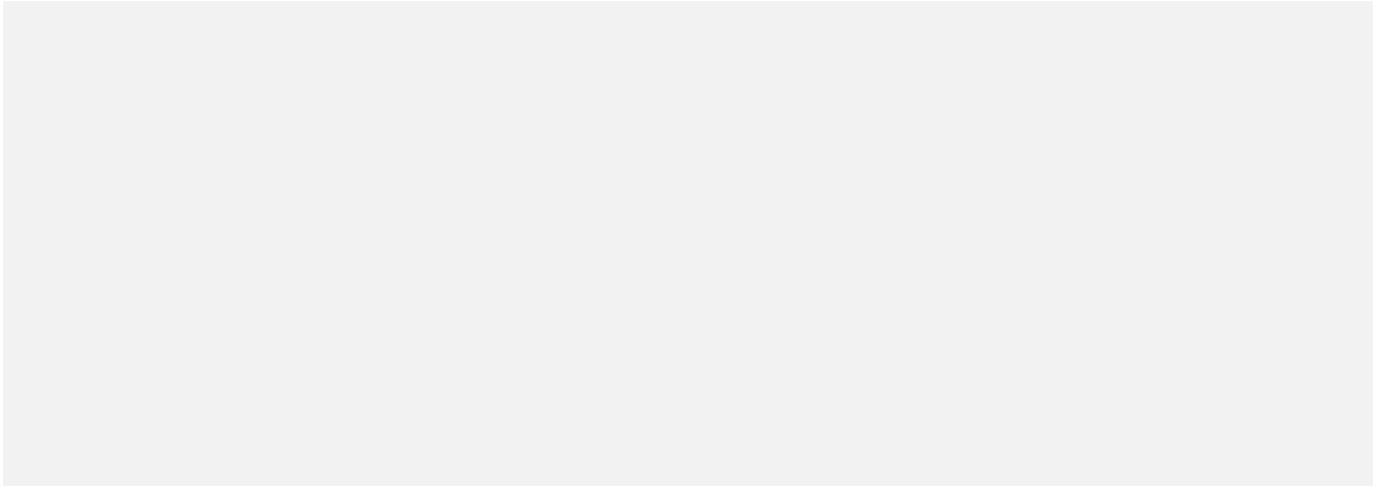
In Reality, Coverage Tools Can Be Wild...

An LLVM-cov Bug Found in Our Work

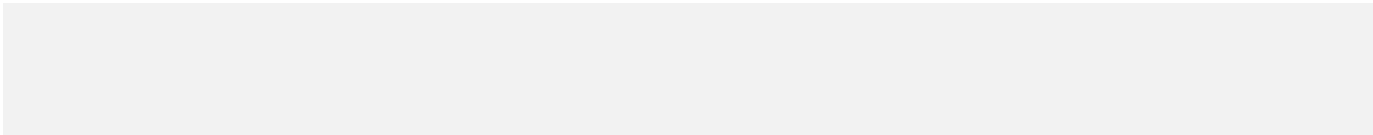
In Reality, Coverage Tools Can Be Wild...

An LLVM-cov Bug Found in Our Work

Snippet under test:



“Test case”:



In Reality, Coverage Tools Can Be Wild...

An LLVM-cov Bug Found in Our Work

Snippet under test:

```
/* Skip IPv6 link-local addresses */  
if (family == AF_INET6) {  
    struct sockaddr_in6 *sin6;  
  
    sin6 = (struct sockaddr_in6 *)ifap->ifa_addr;  
    if (IN6_IS_ADDR_LINKLOCAL(&sin6->sin6_addr) ||  
        IN6_IS_ADDR_MC_LINKLOCAL(&sin6->sin6_addr))  
        continue;  
}
```

“Test case”:

In Reality, Coverage Tools Can Be Wild...

An LLVM-cov Bug Found in Our Work

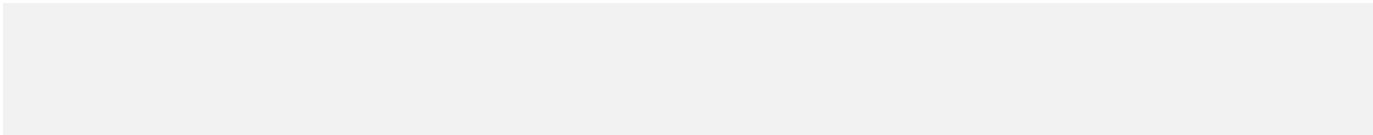
Snippet under test:

```
/* Skip IPv6 link-local addresses */  
if (family == AF_INET6) {  
    struct sockaddr_in6 *sin6;  
  
    sin6 = (struct sockaddr_in6 *)ifap->ifa_addr;  
    if (IN6_IS_ADDR_LINKLOCAL(&sin6->sin6_addr) ||  
        IN6_IS_ADDR_MC_LINKLOCAL(&sin6->sin6_addr))  
        continue;  
}
```


debian

<https://sources.debian.org/src/hostname/3.23%2Bnmu1/hostname.c#L311>

“Test case”:



In Reality, Coverage Tools Can Be Wild...

An LLVM-cov Bug Found in Our Work

Snippet under test:

```
/* Skip IPv6 link-local addresses */  
if (family == AF_INET6) {  
    struct sockaddr_in6 *sin6;  
  
    sin6 = (struct sockaddr_in6 *)ifap->ifa_addr;  
    if (IN6_IS_ADDR_LINKLOCAL(&sin6->sin6_addr) ||  
        IN6_IS_ADDR_MC_LINKLOCAL(&sin6->sin6_addr))  
        continue;  
}
```


debian

<https://sources.debian.org/src/hostname/3.23%2Bnmu1/hostname.c#L311>

“Test case”:

```
$ hostname  
My-Ubuntu-Laptop
```

In Reality, Coverage Tools Can Be Wild...

An LLVM-cov Bug Found in Our Work

Snippet under test:

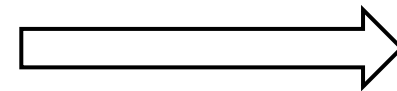
```
/* Skip IPv6 link-local addresses */  
if (family == AF_INET6) {  
    struct sockaddr_in6 *sin6;  
  
    sin6 = (struct sockaddr_in6 *)ifap->ifa_addr;  
    if (IN6_IS_ADDR_LINKLOCAL(&sin6->sin6_addr) ||  
        IN6_IS_ADDR_MC_LINKLOCAL(&sin6->sin6_addr))  
        continue;  
}
```



debian

<https://sources.debian.org/src/hostname/3.23%2Bnmu1/hostname.c#L311>

Coverage
tools



“Test case”:

```
$ hostname  
My-Ubuntu-Laptop
```

In Reality, Coverage Tools Can Be Wild...

An LLVM-cov Bug Found in Our Work

Coverage report for `hostname.c` by LLVM-cov

```
/* Skip IPv6 link-local addresses */
```

```
if (family == AF_INET6) {
```

```
    struct sockaddr_in6 *sin6;
```

```
    sin6 = (struct sockaddr_in6 *)ifap->ifa_addr;
```

```
    if (IN6_IS_ADDR_LINKLOCAL(&sin6->sin6_addr) ||
```

```
        IN6_IS_ADDR_MC_LINKLOCAL(&sin6->sin6_addr))
```

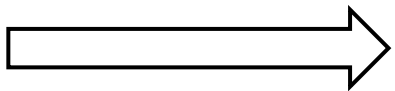
```
        continue;
```

```
}
```



debian

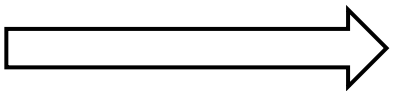
Coverage
tools



In Reality, Coverage Tools Can Be Wild...

An LLVM-cov Bug Found in Our Work

Coverage
tools



Coverage report for hostname.c by LLVM-cov

```
0 | /* Skip IPv6 link-local addresses */  
0 | if (family == AF_INET6) {  
0 |     struct sockaddr_in6 *sin6;  
  
0 |     sin6 = (struct sockaddr_in6 *)ifap->ifa_addr;  
0 |     if (IN6_IS_ADDR_LINKLOCAL(&sin6->sin6_addr) ||  
    |  
0 |     IN6_IS_ADDR_MC_LINKLOCAL(&sin6->sin6_addr))  
    |  
0 |     continue;  
0 | }
```

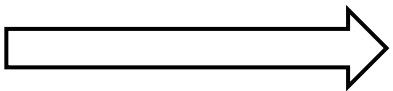


debian

In Reality, Coverage Tools Can Be Wild...

An LLVM-cov Bug Found in Our Work

Coverage
tools



Coverage report for hostname.c by LLVM-cov

```
0 | /* Skip IPv6 link-local addresses */
0 | if (family == AF_INET6) {
0 |     struct sockaddr_in6 *sin6;
0 |
0 |     sin6 = (struct sockaddr_in6 *)ifap->ifa_addr;
0 |     if (IN6_IS_ADDR_LINKLOCAL(&sin6->sin6_addr) ||
    [True: 0, False: 0]
0 |         IN6_IS_ADDR_MC_LINKLOCAL(&sin6->sin6_addr))
    [ ]
0 |         continue;
0 | }
```

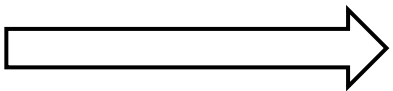


debian

In Reality, Coverage Tools Can Be Wild...

An LLVM-cov Bug Found in Our Work

Coverage
tools



Coverage report for hostname.c by LLVM-cov

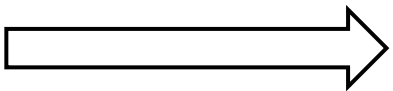
```
0 | /* Skip IPv6 link-local addresses */  
0 | if (family == AF_INET6) {  
0 |     struct sockaddr_in6 *sin6;  
0 |  
0 |     sin6 = (struct sockaddr_in6 *)ifap->ifa_addr;  
0 |     if (IN6_IS_ADDR_LINKLOCAL(&sin6->sin6_addr) ||  
[True: 0, False: 0]  
0 |         IN6_IS_ADDR_MC_LINKLOCAL(&sin6->sin6_addr))  
[True: 18446744073709551615, False: 1]  
0 |         continue;  
0 | }
```



In Reality, Coverage Tools Can Be Wild...

An LLVM-cov Bug Found in Our Work

Coverage
tools



Coverage report for hostname.c by LLVM-cov

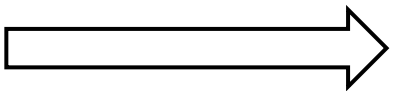
```
0 | /* Skip IPv6 link-local addresses */
0 | if (family == AF_INET6) {
0 |     struct sockaddr_in6 *sin6;
0 |
0 |     sin6 = (struct sockaddr_in6 *)ifap->ifa_addr;
0 |     if (IN6_IS_ADDR_LINKLOCAL(&sin6->sin6_addr) ||
    [True: 0, False: 0]
0 |         IN6_IS_ADDR_MC_LINKLOCAL(&sin6->sin6_addr))
    [True: 18446744073709551615, False: 1]
0 |         continue;
0 | }
```



In Reality, Coverage Tools Can Be Wild...

An LLVM-cov Bug Found in Our Work

Coverage
tools



Coverage report for hostname.c by LLVM-cov

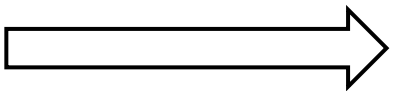
```
0 | /* Skip IPv6 link-local addresses */  
0 | if (family == AF_INET6) {  
0 |     struct sockaddr_in6 *sin6;  
  
0 |     sin6 = (struct sockaddr_in6 *)ifap->ifa_addr;  
0 |     if (IN6_IS_ADDR_LINKLOCAL(&sin6->sin6_addr) ||  
[True: 0, False: 0]  
0 |         IN6_IS_ADDR_MC_LINKLOCAL(&sin6->sin6_addr))  
[True: 18446744073709551615, False: 1]  
0 |         continue;  
0 | }
```



In Reality, Coverage Tools Can Be Wild...

An LLVM-cov Bug Found in Our Work

Coverage
tools



Coverage report for hostname.c by LLVM-cov

```
0 | /* Skip IPv6 link-local addresses */  
0 | if (family == AF_INET6) {  
0 |     struct sockaddr_in6 *sin6;  
  
0 |     sin6 = (struct sockaddr_in6 *)ifap->ifa_addr;  
0 |     if (IN6_IS_ADDR_LINKLOCAL(&sin6->sin6_addr) ||
```

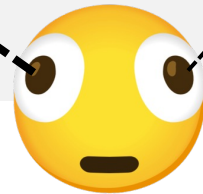


[True: 0, False: 0]

```
0 |     IN6_IS_ADDR_MC_LINKLOCAL(&sin6->sin6_addr))
```

[True: 18446744073709551615, False: 1]

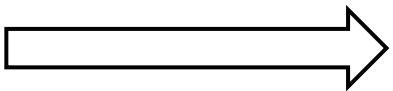
```
0 |     continue;  
0 | }
```



In Reality, Coverage Tools Can Be Wild...

An LLVM-cov Bug Found in Our Work

Coverage
tools



Coverage report for hostname.c by LLVM-cov

```
0 | /* Skip IPv6 link-local addresses */
0 | if (family == AF_INET6) {
    struct sockaddr_in6 *sin6;
    sin6 = (struct sockaddr_in6 *)ifap->ifa_addr;
    if (IN6_IS_ADDR_LINKLOCAL(&sin6->sin6_addr) ||
```

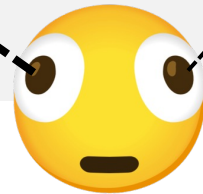


[True: 0, False: 0]

```
0 | IN6_IS_ADDR_MC_LINKLOCAL(&sin6->sin6_addr))
```

[True: 18446744073709551615, False: 1]

```
0 | continue;
0 | }
```

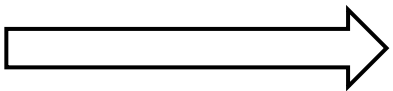


1.84E = 1.84×10^{18}

In Reality, Coverage Tools Can Be Wild...

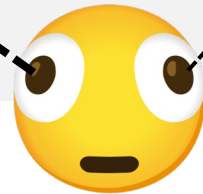
An LLVM-cov Bug Found in Our Work

Coverage
tools



Coverage report for hostname.c by LLVM-cov

```
0 | /* Skip IPv6 link-local addresses */
0 | if (family == AF_INET6) {
0 |     struct sockaddr_in6 *sin6;
0 |
0 |     sin6 = (struct sockaddr_in6 *)ifap->ifa_addr;
0 |     if (IN6_IS_ADDR_LINKLOCAL(&sin6->sin6_addr) ||
    [True: 0, False: 0]
0 |         IN6_IS_ADDR_MC_LINKLOCAL(&sin6->sin6_addr))
    [True: 18446744073709551615, False: 1]
0 |         continue;
0 | }
```



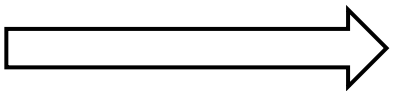
$1.84\text{E} = 1.84 \times 10^{18}$

An Intel Core i9-14900k (5 GHz) would need **117 years** to print your hostname!

In Reality, Coverage Tools Can Be Wild...

An LLVM-cov Bug Found in Our Work

Coverage
tools



Coverage report for hostname.c by LLVM-cov

```
0 | /* Skip IPv6 link-local addresses */  
0 | if (family == AF_INET6) {  
0 |     struct sockaddr_in6 *sin6;  
  
0 |     sin6 = (struct sockaddr_in6 *)ifap->ifa_addr;  
0 |     if (IN6_IS_ADDR_LINKLOCAL(&sin6->sin6_addr) ||
```



[True: 0, False: 0]

```
0 |     IN6_IS_ADDR_MC_LINKLOCAL(&sin6->sin6_addr))
```

[True: 18446744073709551615, False: 1]



```
0 |     continue;  
0 | }
```

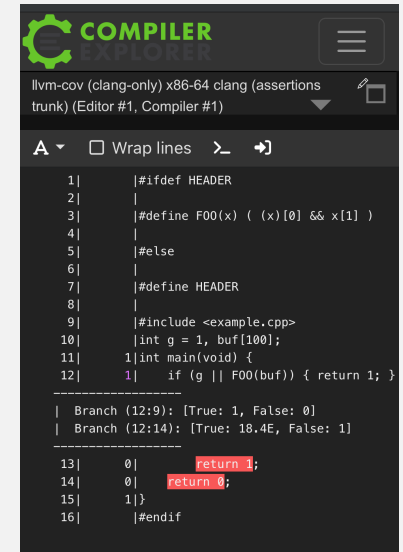
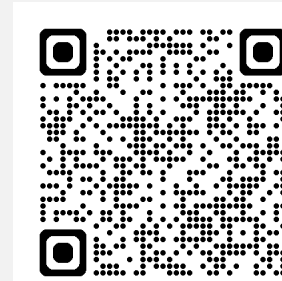


Deeper into This LLVM-cov Bug

- ① Original `hostname.c` of ~500 LoC

You can play with it on your phone!

<https://godbolt.org/z/7W3G61zT4>



```
COMPILER EXPLORER
llvm-cov (clang-only) x86-64 clang (assertions)
trunk (Editor #1, Compiler #1)

A ▾ □ Wrap lines >_ ↵

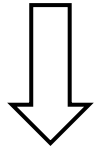
1|  |#ifdef HEADER
2|  |
3|  |#define F00(x) ( (x)[0] && x[1] )
4|  |
5|  |#else
6|  |
7|  |#define HEADER
8|  |
9|  |#include <example.cpp>
10| |int g = 1, buf[100];
11| |int main(void) {
12| |    if (g || F00(buf)) { return 1; }

| Branch (12:9): [True: 1, False: 0]
| Branch (12:14): [True: 18.4E, False: 1]

13| |    return 1;
14| |    return 0;
15| |}
16| |#endif
```

Deeper into This LLVM-cov Bug

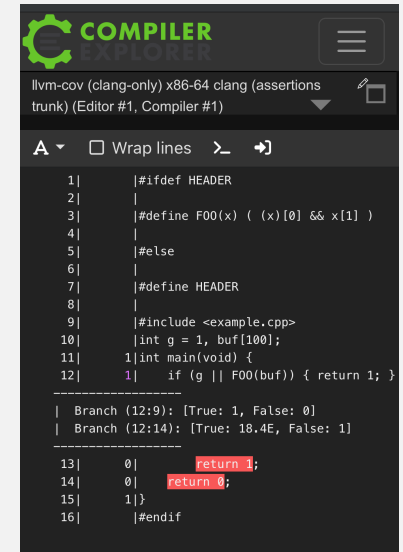
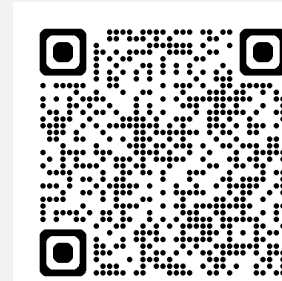
① Original `hostname.c` of ~500 LoC



② Smaller program of 7 LoC retaining the symptom

You can play with it on your phone!

<https://godbolt.org/z/7W3G61zT4>



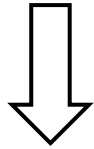
```
1|  |#ifdef HEADER
2|  |
3|  |#define F00(x) ( (x)[0] && x[1] )
4|  |
5|  |#else
6|  |
7|  |#define HEADER
8|  |
9|  |#include <example.cpp>
10| int g = 1, buf[100];
11| int main(void) {
12|     if (g || F00(buf)) { return 1; }
13| }
14|
15| }
16| #endif
```

Branch (12:9): [True: 1, False: 0]
Branch (12:14): [True: 18.4E, False: 1]

return 1;
return 0;

Deeper into This LLVM-cov Bug

- ① Original `hostname.c` of ~500 LoC



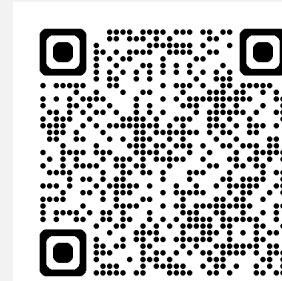
- ② Smaller program of 7 LoC retaining the symptom

`test.h`

```
#define F00(x) ((x)[0] && x[1])
```

You can play with it on your phone!

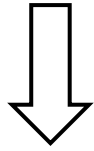
<https://godbolt.org/z/7W3G61zT4>



```
COMPILER EXPLORER
llvm-cov (clang-only) x86-64 clang (assertions)
trunk (Editor #1, Compiler #1)
A □ Wrap lines > ↵
1|  |#ifdef HEADER
2|  |
3|  |#define F00(x) ( (x)[0] && x[1] )
4|  |
5|  |#else
6|  |
7|  |#define HEADER
8|  |
9|  |#include <example.cpp>
10| int g = 1, buf[100];
11| int main(void) {
12|     if (g || F00(buf)) { return 1; }
13|
14|     return 1;
15| }
16| #endif
| Branch (12:9): [True: 1, False: 0]
| Branch (12:14): [True: 18.4E, False: 1]
13|     0|     return 1;
14|     0|     return 0;
15|     1|
16|     #endif
```


Deeper into This LLVM-cov Bug

- ① Original `hostname.c` of ~500 LoC



- ② Smaller program of 7 LoC retaining the symptom

`test.h`

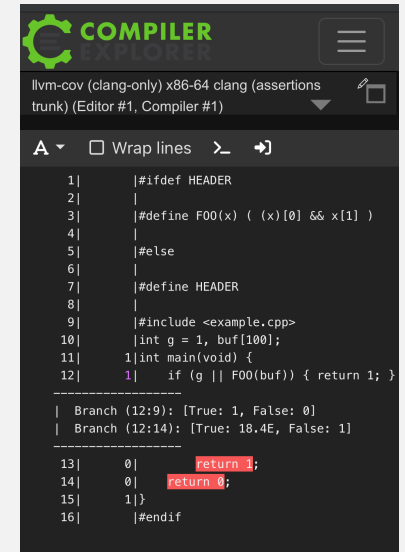
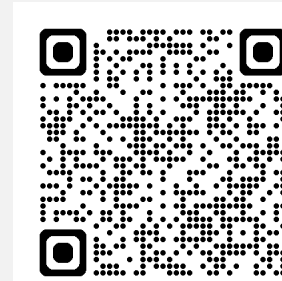
```
#define F00(x) ((x)[0] && x[1])
```

`test.c`

```
#include <test.h>
int g = 1, buf[100];
int main(void) {
    if (g || F00(buf)) { return 1; }
    return 0;
}
```

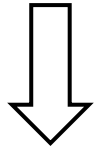
You can play with it on your phone!

<https://godbolt.org/z/7W3G61zT4>



Deeper into This LLVM-cov Bug

- ① Original `hostname.c` of ~500 LoC



- ② Smaller program of 7 LoC retaining the symptom

`test.h`

```
#define F00(x) ((x)[0] && x[1])
```

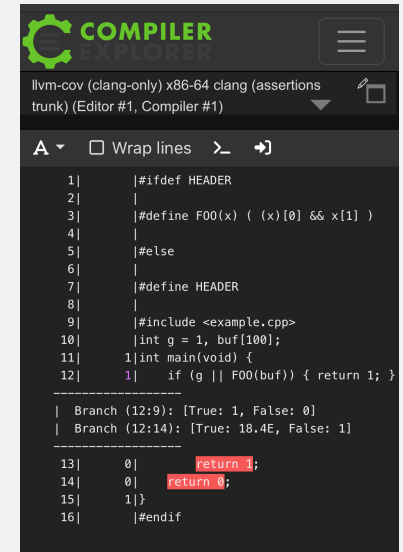
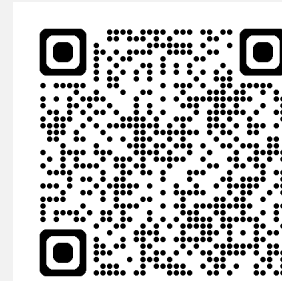
`test.c`

```
#include <test.h>
int g = 1, buf[100];
int main(void) {
    if (g || F00(buf)) { return 1; }
    return 0;
}
```

1. a macro from system header

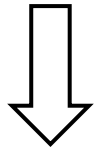
You can play with it on your phone!

<https://godbolt.org/z/7W3G61zT4>



Deeper into This LLVM-cov Bug

- ① Original `hostname.c` of ~500 LoC



- ② Smaller program of 7 LoC retaining the symptom

`test.h`

```
#define F00(x) ((x)[0] && x[1])
```

`test.c`

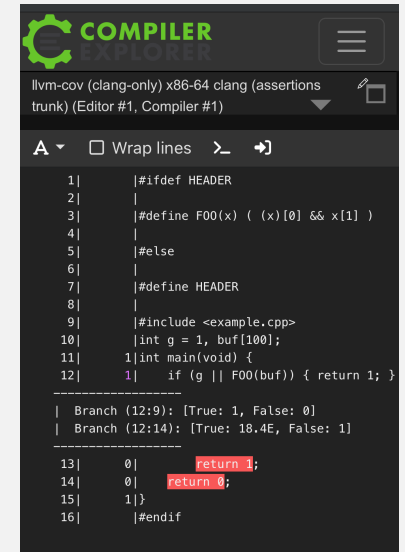
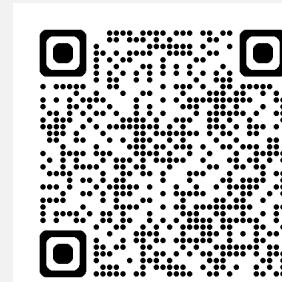
```
#include <test.h>
int g = 1, buf[100];
int main(void) {
    if (g || F00(buf)) { return 1; }
    return 0;
}
```

2. the macro subscripts the argument in parentheses

1. a macro from system header

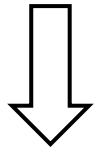
You can play with it on your phone!

<https://godbolt.org/z/7W3G61zT4>



Deeper into This LLVM-cov Bug

- ① Original `hostname.c` of ~500 LoC



- ② Smaller program of 7 LoC retaining the symptom

`test.h`

```
#define F00(x) ((x)[0] && x[1])
```

`test.c`

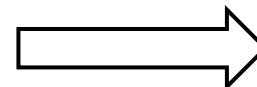
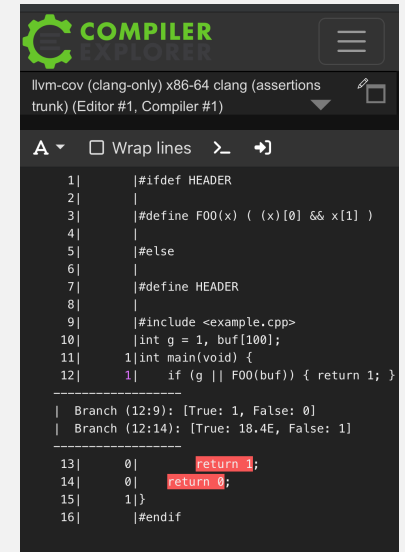
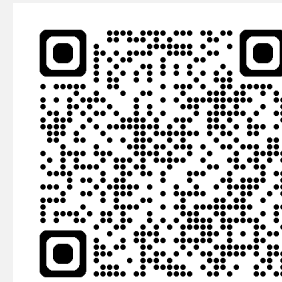
```
#include <test.h>
int g = 1, buf[100];
int main(void) {
    if (g || F00(buf)) { return 1; }
    return 0;
}
```

1. a macro from system header

2. the macro subscripts the argument in parentheses

You can play with it on your phone!

<https://godbolt.org/z/7W3G61zT4>



- ③ Reported as LLVM#131505 by our work

Main Prior Work (“C2V”)

- [Yang et al. 2019] proposed **differential testing** of Gcov and LLVM-cov, the most widely used coverage tools for C/C++

Main Prior Work (“C2V”)

- [Yang et al. 2019] proposed **differential testing** of Gcov and LLVM-cov, the most widely used coverage tools for C/C++
 - **Random programs** generated by Csmith used as test inputs

Main Prior Work (“C2V”)

- [Yang et al. 2019] proposed **differential testing** of Gcov and LLVM-cov, the most widely used coverage tools for C/C++
 - **Random programs** generated by Csmith used as test inputs

```
l_1841 ^=
  (l_1630[g_269] <
   (l_1630[(g_176.f3 + 2)] ||
    (((safe_mod_func_int32_t_s_s(
      ((*g_554) = (0x8CFE1A | (*l_1664))),
      (0x589B |
       (safe_unary_minus_func_int64_t_s(
         ((safe_div_func_int32_t_s_s(((void *)0 != l_1836),
                                     p_28)) ^
          ((*l_1623) =
           ((*g_165) = (safe_rshift_func_int16_t_s_s(
             0x1B78L, 5))) > p_28))),
          (-1L)))) <= l_1839)))) <= 18446744073709551613UL) !=
   p_28));
```



A relatively “small” statement generated by Csmith

Main Prior Work (“C2V”)

- [Yang et al. 2019] proposed **differential testing** of Gcov and LLVM-cov, the most widely used coverage tools for C/C++
 - **Random programs** generated by Csmith used as test inputs
 - Output: lines where Gcov and LLVM-cov report different line coverage

```
l_1841 ^=
  (l_1630[g_269] <
   (l_1630[(g_176.f3 + 2)] ||
    (((safe_mod_func_int32_t_s_s(
      (((**g_554) = (0x8CFE1A | (*l_1664))),
      (0x589BL |
        (safe_unary_minus_func_int64_t_s(
          (((safe_div_func_int32_t_s_s(((void *)0 != l_1836),
                                         p_28)) ^
            ((**l_1623) =
              (((**g_165) = (safe_rshift_func_int16_t_s_s(
                0x1B78L, 5))) > p_28))),
            (-1L)))) <= l_1839)))) <= 18446744073709551613UL) !=
    p_28))));
```



A relatively “small” statement generated by Csmith

Main Prior Work (“C2V”)

- [Yang et al. 2019] proposed **differential testing** of Gcov and LLVM-cov, the most widely used coverage tools for C/C++
 - **Random programs** generated by Csmith used as test inputs
 - Output: lines where Gcov and LLVM-cov report different line coverage
- Found lots of bugs but still missed the bug exposed with `hostname.c`

```
l_1841 ^=
  (l_1630[g_269] <
   (l_1630[(g_176.f3 + 2)] ||
    (((safe_mod_func_int32_t_s_s(
      (((**g_554) = (0x8CFEFE1AL | (*l_1664))),
      (0x589BL |
       ((safe_unary_minus_func_int64_t_s(
         (((safe_div_func_int32_t_s_s(((void *)0 != l_1836),
                                       p_28))) ^
          ((**l_1623) =
            (((**g_165) = (safe_rshift_func_int16_t_s_s(
              0x1B78L, 5))) > p_28))),
          (-1L)))) <= l_1839)))) <= 18446744073709551613UL) !=
   p_28))));
```



A relatively “small” statement generated by Csmith

Main Prior Work (“C2V”)

- [Yang et al. 2019] proposed **differential testing** of Gcov and LLVM-cov, the most widely used coverage tools for C/C++
 - **Random programs** generated by Csmith used as test inputs
 - Output: lines where Gcov and LLVM-cov report different line coverage
- Found lots of bugs but still missed the bug exposed with `hostname.c`

```
l_1841 ^=
    (l_1630[g_269] <
    (l_1630[(g_176.f3 + 2)] ||
    (((safe_mod_func_int32_t_s_s(
        (*****g_554) = (0x8CFE1A1 | (*l_1664))),
        (0x589BL |
        ((safe_unary_minus_func_int64_t_s(
            (((safe_div_func_int32_t_s_s(((v
                p_2
                ((*l_1623) =
                ((*g_165) = (safe_rsh
                    0x1B78L, 5))) > p_2
                (-1L)))) <= l_1839)))) <= 18446744073709551613UL) !=
                p_2)))));
```



Csmith

Though complex, not capturing
real-world code patterns

A relatively “small” statement generated by Csmith

DEBCovDIFF: Contributions

1. Key insight: **real-world software** used as test inputs for coverage tools

DEBCOVDIFF: Contributions

1. Key insight: **real-world software** used as test inputs for coverage tools
 - Classic **differential testing** as in prior work C2V

DEBCOVDIFF: Contributions

1. Key insight: **real-world software** used as test inputs for coverage tools
 - Classic **differential testing** as in prior work C2V
2. Mitigate program execution **nondeterminism**

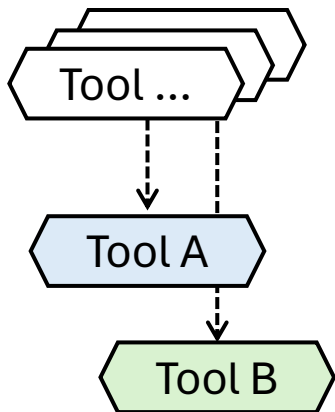
DEBCOVDIFF: Contributions

1. Key insight: **real-world software** used as test inputs for coverage tools
 - Classic **differential testing** as in prior work C2V
2. Mitigate program execution **nondeterminism**
3. Include advanced metrics, i.e., **branch coverage** and **MC/DC**

DEBCOVDIFF: Contributions

1. Key insight: **real-world software** used as test inputs for coverage tools
 - Classic **differential testing** as in prior work C2V
2. Mitigate program execution **nondeterminism**
3. Include advanced metrics, i.e., **branch coverage** and **MC/DC**

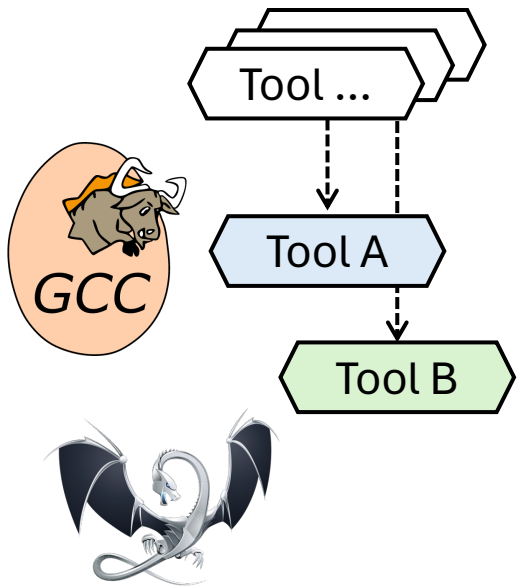
Study Subjects



DEBCOVDIFF: Contributions

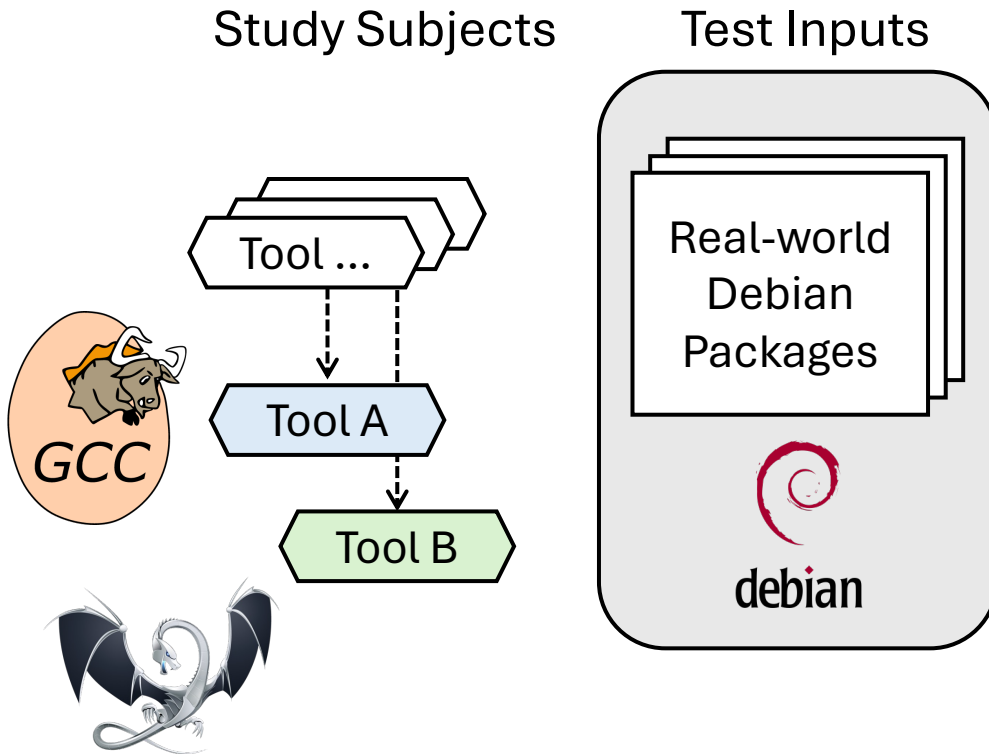
1. Key insight: **real-world software** used as test inputs for coverage tools
 - Classic **differential testing** as in prior work C2V
2. Mitigate program execution **nondeterminism**
3. Include advanced metrics, i.e., **branch coverage** and **MC/DC**

Study Subjects



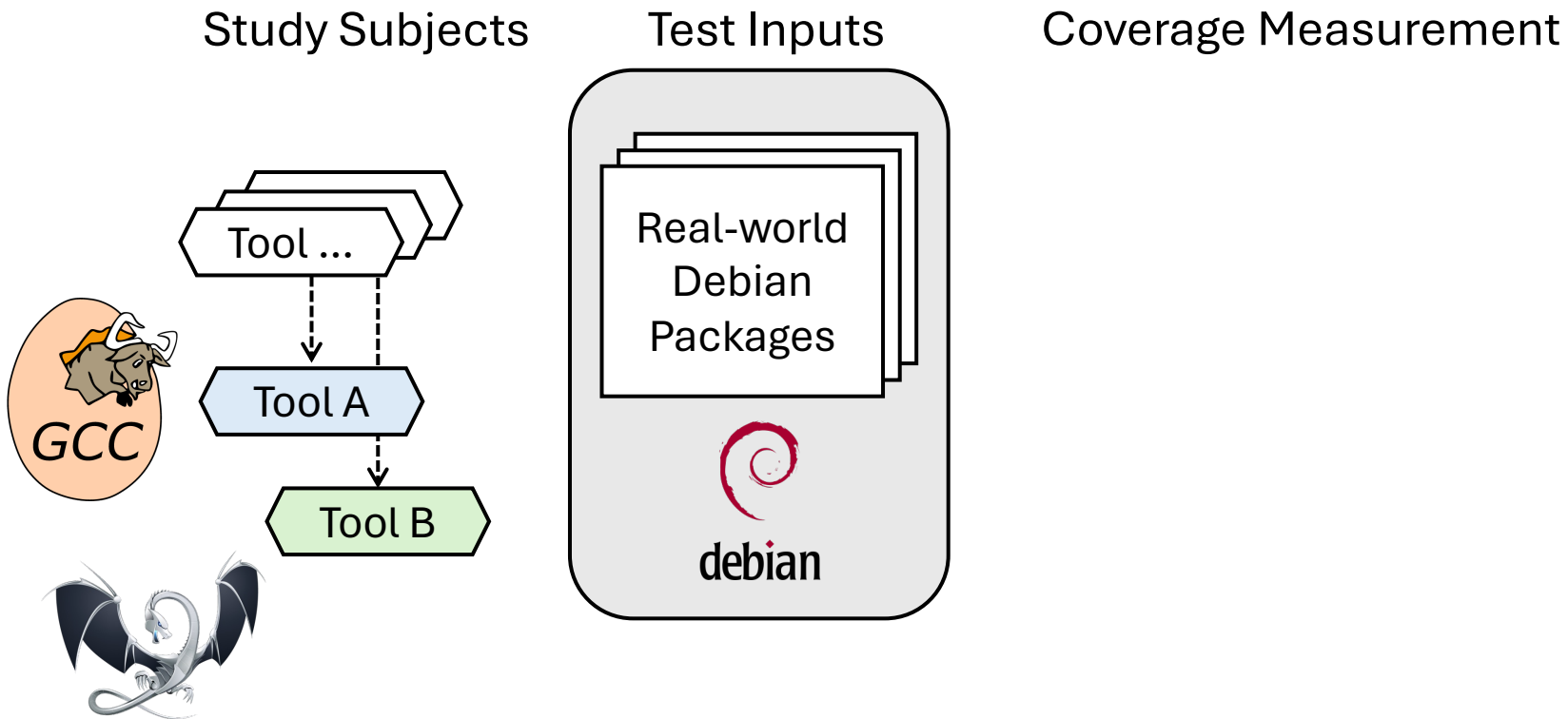
DEBCOVDIFF: Contributions

1. Key insight: **real-world software** used as test inputs for coverage tools
 - Classic **differential testing** as in prior work C2V
2. Mitigate program execution **nondeterminism**
3. Include advanced metrics, i.e., **branch coverage** and **MC/DC**



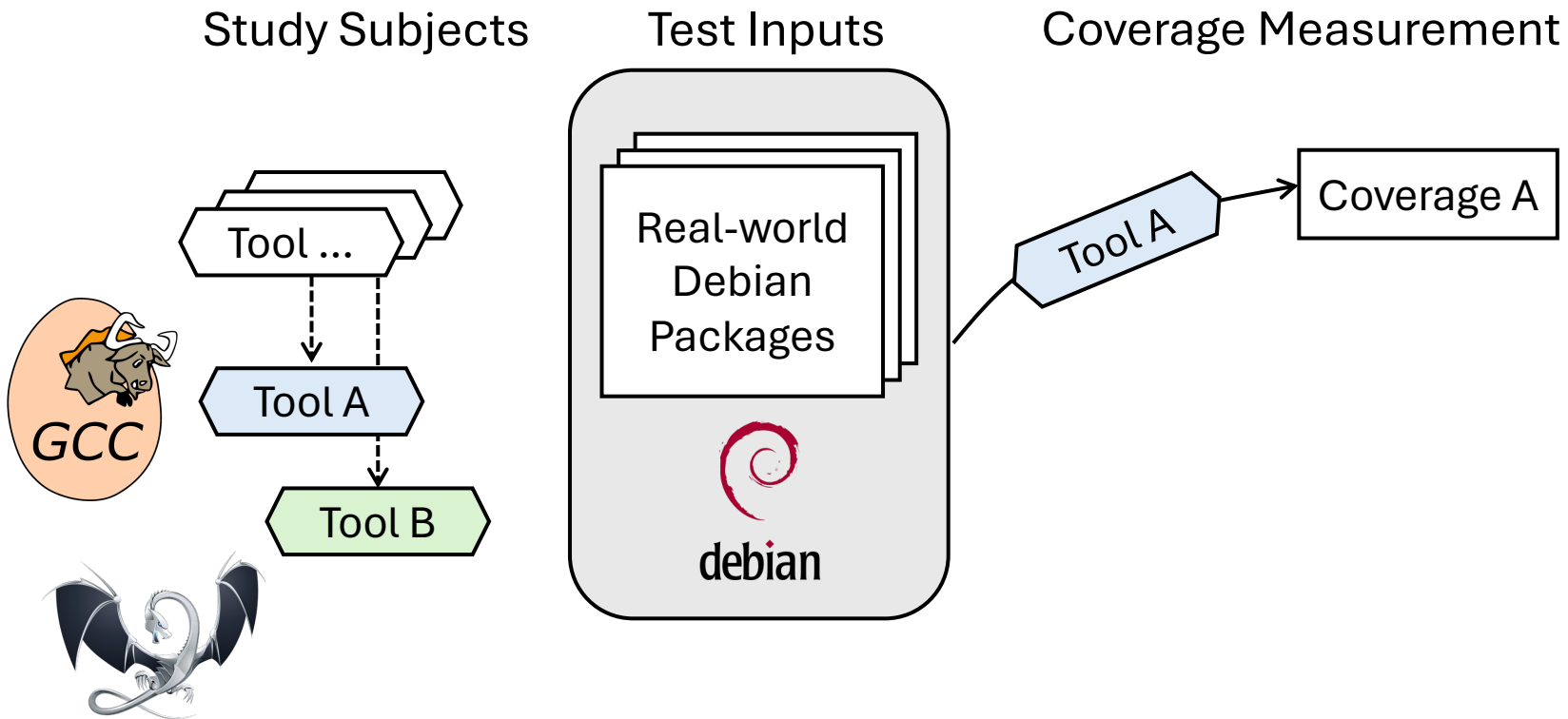
DEBCOVDIFF: Contributions

1. Key insight: **real-world software** used as test inputs for coverage tools
 - Classic **differential testing** as in prior work C2V
2. Mitigate program execution **nondeterminism**
3. Include advanced metrics, i.e., **branch coverage** and **MC/DC**



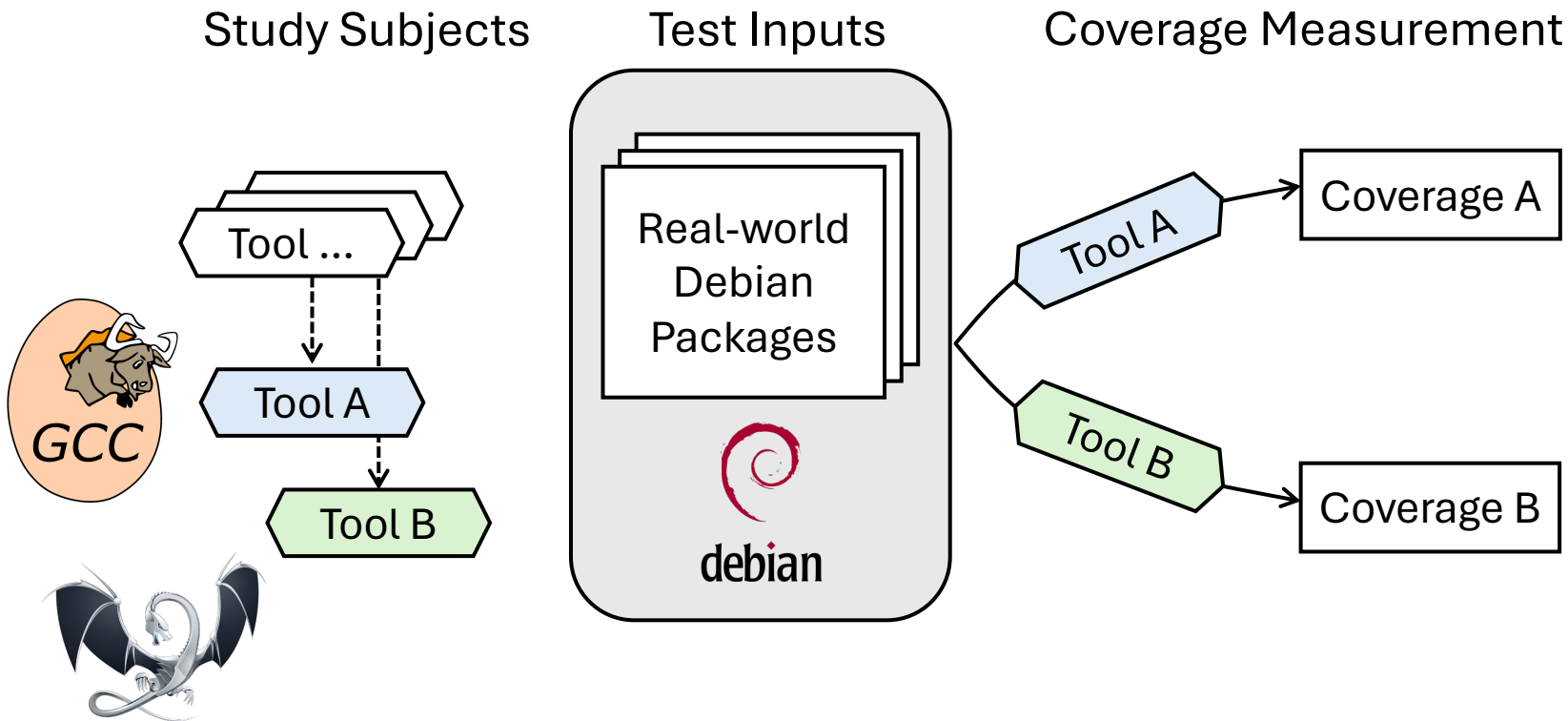
DEBCOVDIFF: Contributions

1. Key insight: **real-world software** used as test inputs for coverage tools
 - Classic **differential testing** as in prior work C2V
2. Mitigate program execution **nondeterminism**
3. Include advanced metrics, i.e., **branch coverage** and **MC/DC**



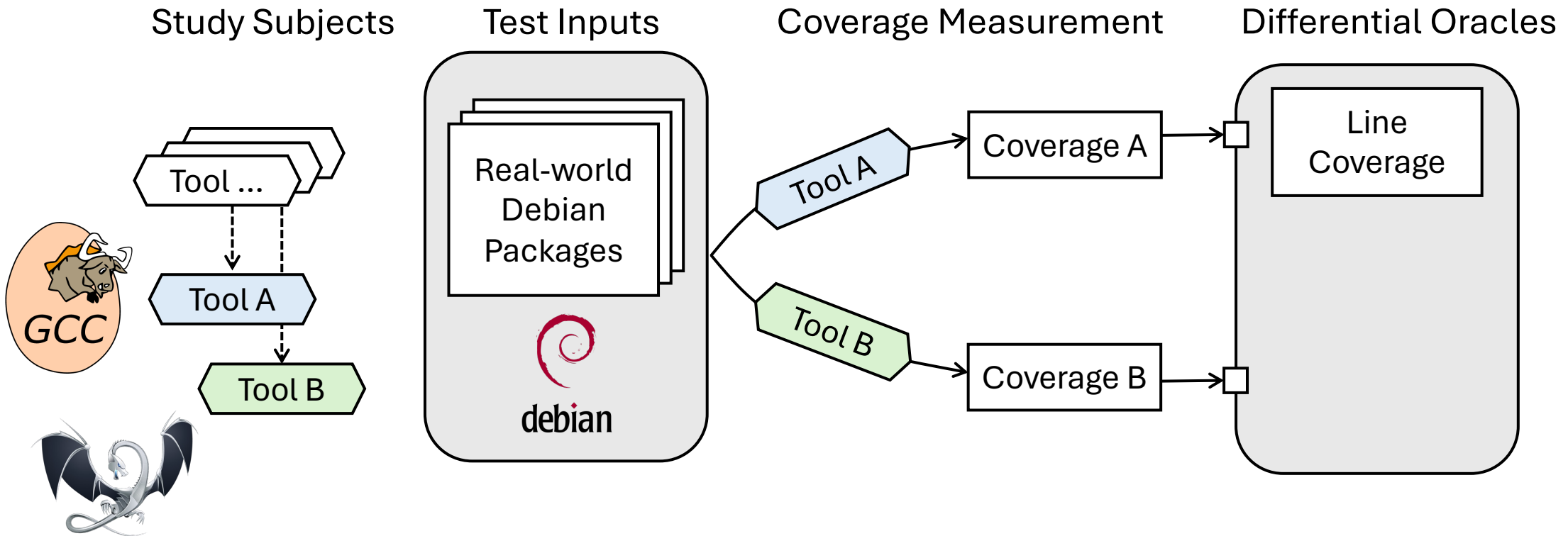
DEBCOVDIFF: Contributions

1. Key insight: **real-world software** used as test inputs for coverage tools
 - Classic **differential testing** as in prior work C2V
2. Mitigate program execution **nondeterminism**
3. Include advanced metrics, i.e., **branch coverage** and **MC/DC**



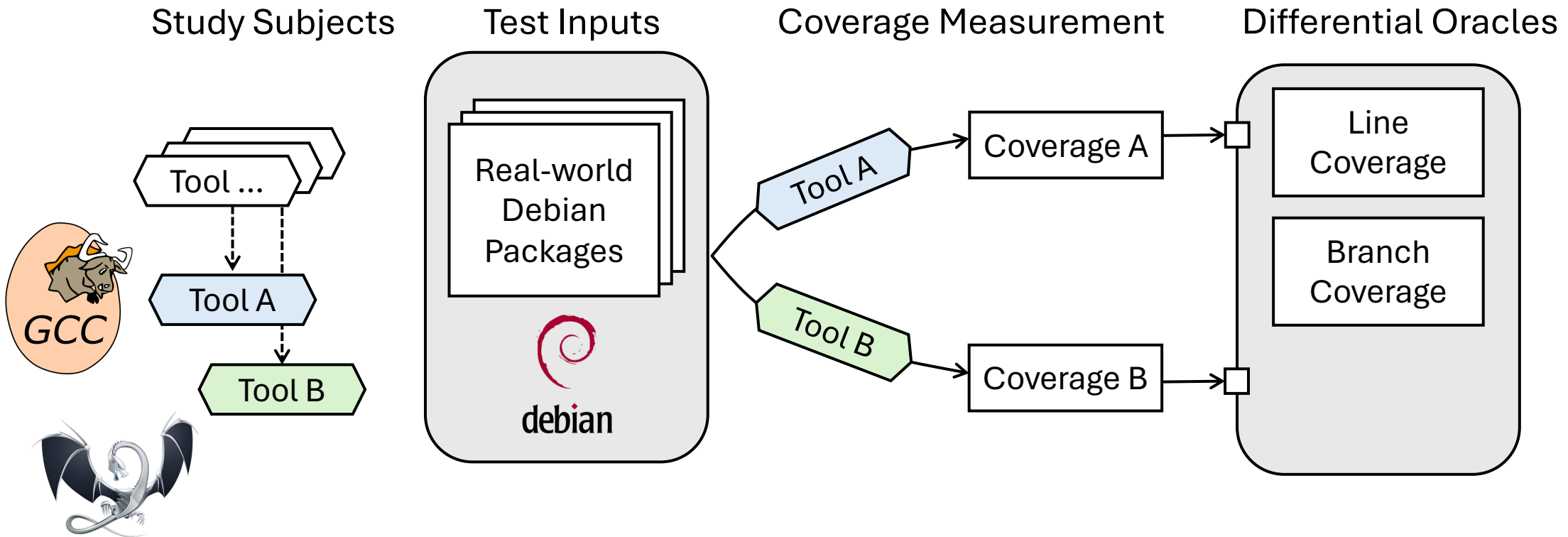
DEBCOVDIFF: Contributions

1. Key insight: **real-world software** used as test inputs for coverage tools
 - Classic **differential testing** as in prior work C2V
2. Mitigate program execution **nondeterminism**
3. Include advanced metrics, i.e., **branch coverage** and **MC/DC**



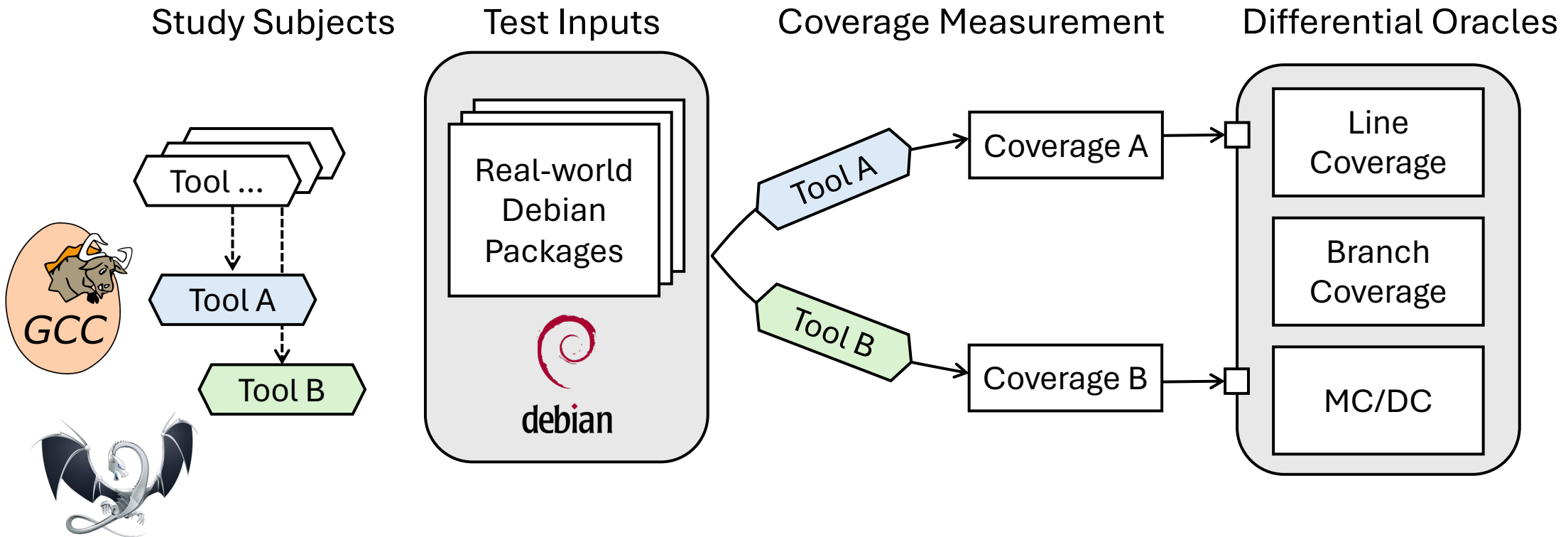
DEBCOVDIFF: Contributions

1. Key insight: **real-world software** used as test inputs for coverage tools
 - Classic **differential testing** as in prior work C2V
2. Mitigate program execution **nondeterminism**
3. Include advanced metrics, i.e., **branch coverage** and **MC/DC**



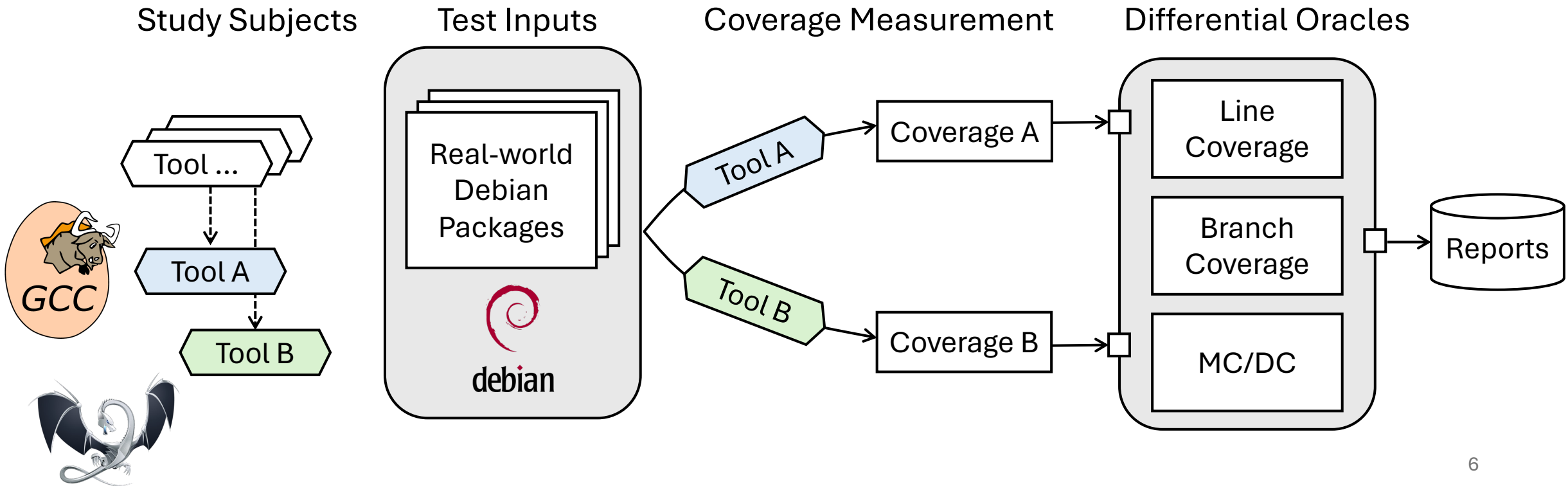
DEBCovDIFF: Contributions

1. Key insight: **real-world software** used as test inputs for coverage tools
 - Classic **differential testing** as in prior work C2V
2. Mitigate program execution **nondeterminism**
3. Include advanced metrics, i.e., **branch coverage** and **MC/DC**



DEBCOVDIFF: Contributions

1. Key insight: **real-world software** used as test inputs for coverage tools
 - Classic **differential testing** as in prior work C2V
2. Mitigate program execution **nondeterminism**
3. Include advanced metrics, i.e., **branch coverage** and **MC/DC**



DEBCovDIFF: Contributions

1. Key insight: **real-world software** used as test inputs for coverage tools
 - Classic **differential testing** as in prior work C2V
2. Mitigate program execution **nondeterminism**
3. Include advanced metrics, i.e., **branch coverage** and **MC/DC**

Study Subjects

Test Inputs

Coverage Measurement

Differential Oracles



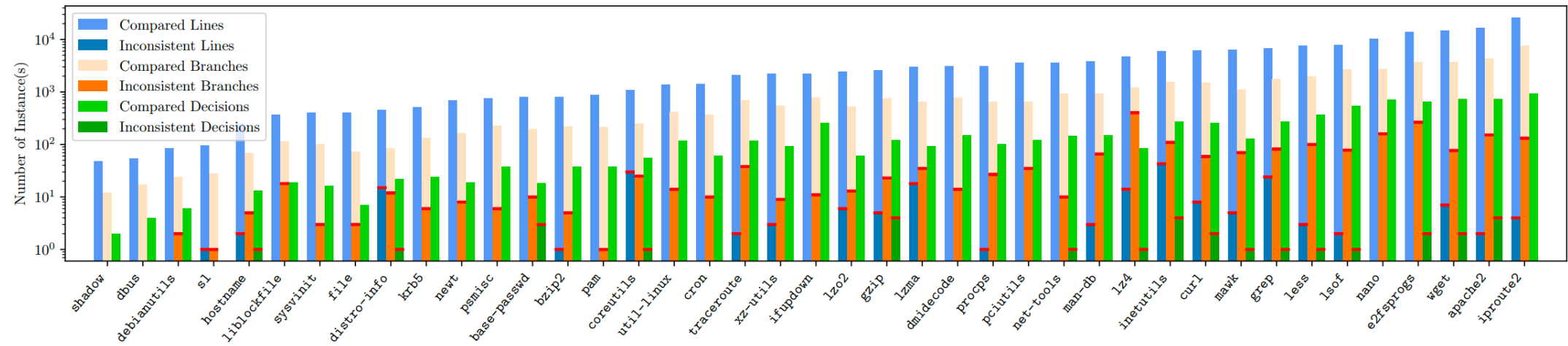
```
- Branch coverage: inconsistency at show_name:hostname-3.23+nmul/hostname.c:313
- number of outcome(s) reported by GCC is 4
- number of outcome(s) reported by LLVM is 2
-
```

**The hostname
case is caught
by DEBCovDIFF!**

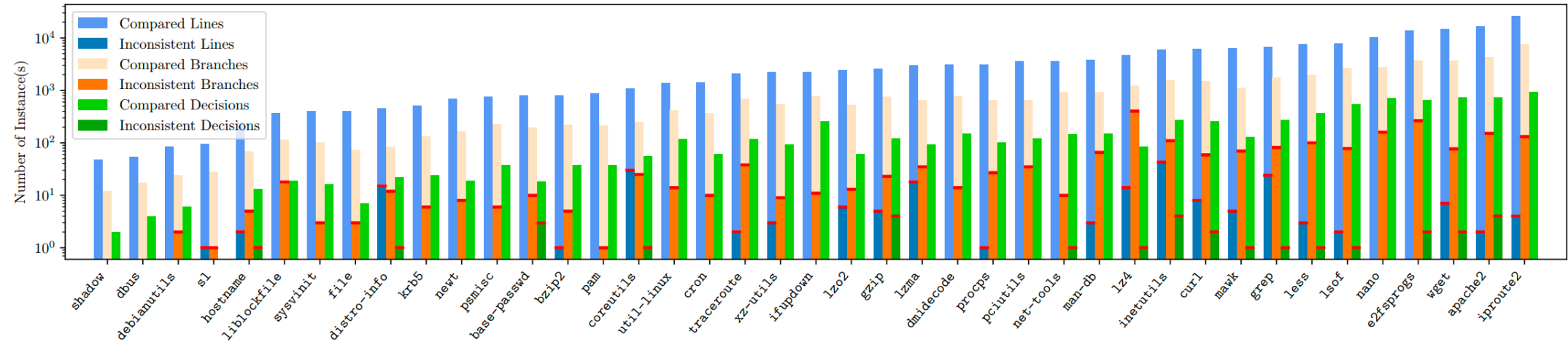
```
- 305 |                                     sizeof(struct sockaddr_in6);
- 306 |
- 307 |                                     /* Skip IPv6 link-local addresses */
- 308 |                                     if (family == AF_INET6) {
- 309 |                                         struct sockaddr_in6 *sin6;
- 310 |
- 311 |                                         sin6 = (struct sockaddr_in6 *)ifap->ifa_addr;
- 312 |                                         if (IN6_IS_ADDR_LINKLOCAL(&sin6->sin6_addr) ||
- 313 |                                             IN6_IS_ADDR_MC_LINKLOCAL(&sin6->sin6_addr))
- 314 |                                             continue;
- 315 |                                     }
```

Reports

Inconsistencies for Simple Commands

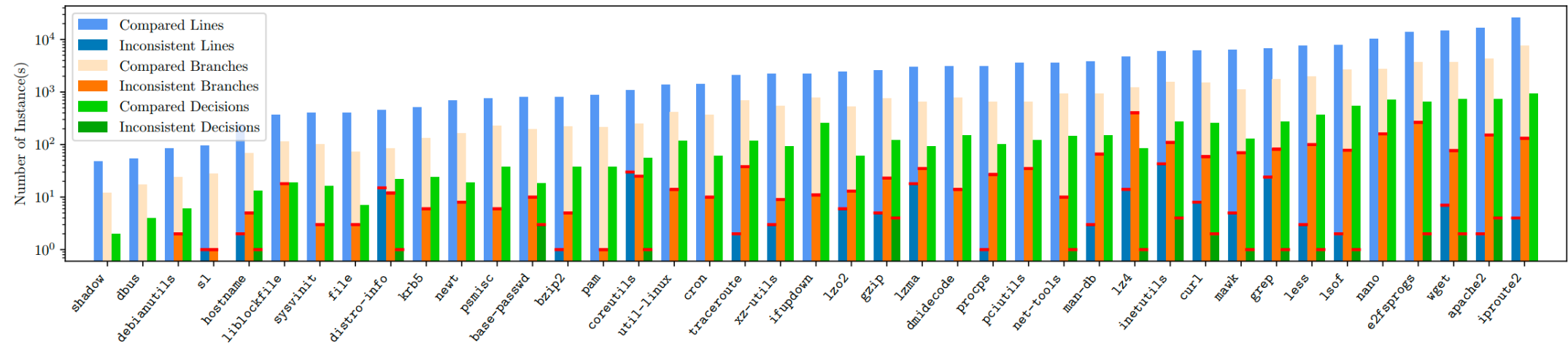


Inconsistencies for Simple Commands



	Lines	Branches	MC/DC
Total compared	168,549	44,126	7,527
Total inconsistent (number + value)	199	2,098 (1,884 + 214)	30 (29 + 1)

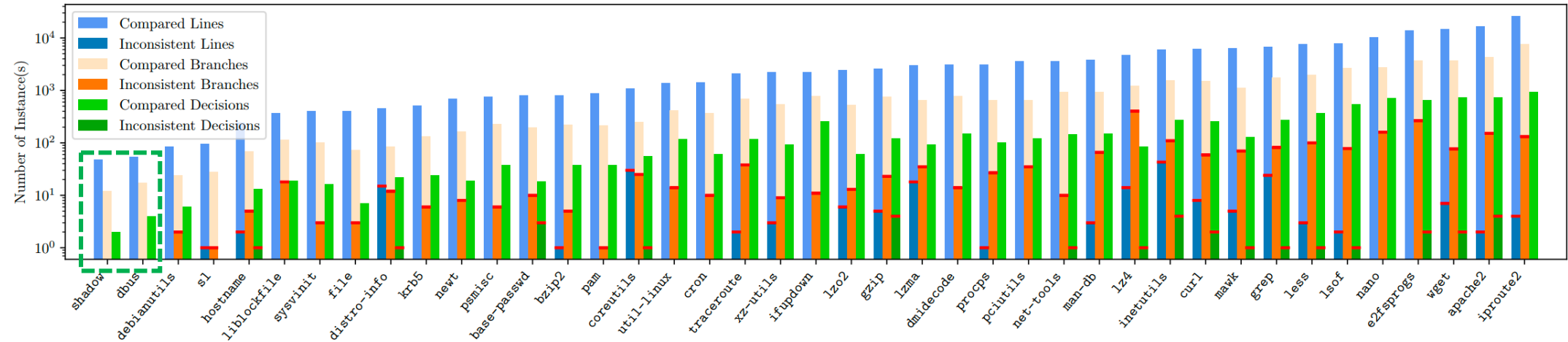
Inconsistencies for Simple Commands



	Lines	Branches	MC/DC
Total compared	168,549	44,126	7,527
Total inconsistent (number + value)	199	2,098 (1,884 + 214)	30 (29 + 1)

- **39** packages exhibit in total **2,327** inconsistencies

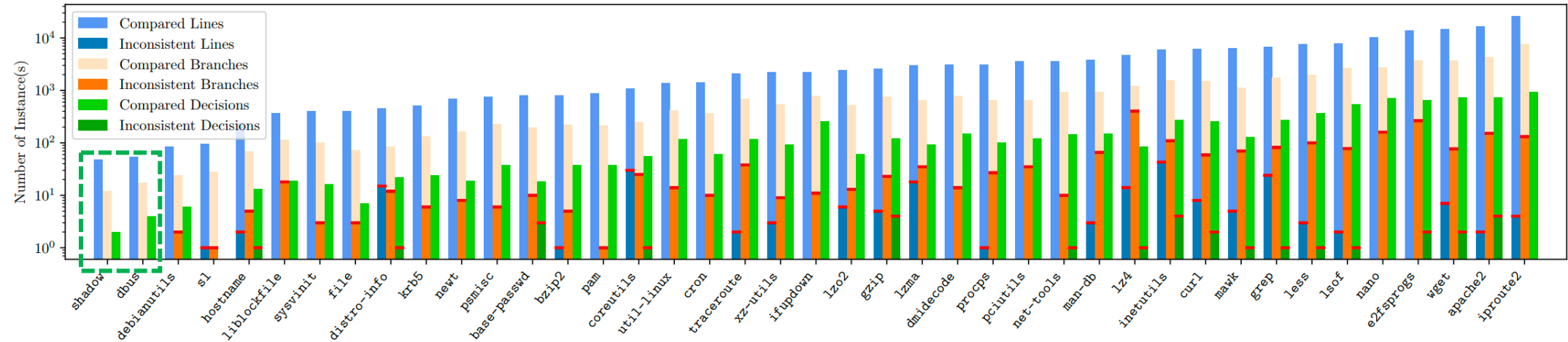
Inconsistencies for Simple Commands



	Lines	Branches	MC/DC
Total compared	168,549	44,126	7,527
Total inconsistent (number + value)	199	2,098 (1,884 + 214)	30 (29 + 1)

- **39** packages exhibit in total **2,327** inconsistencies

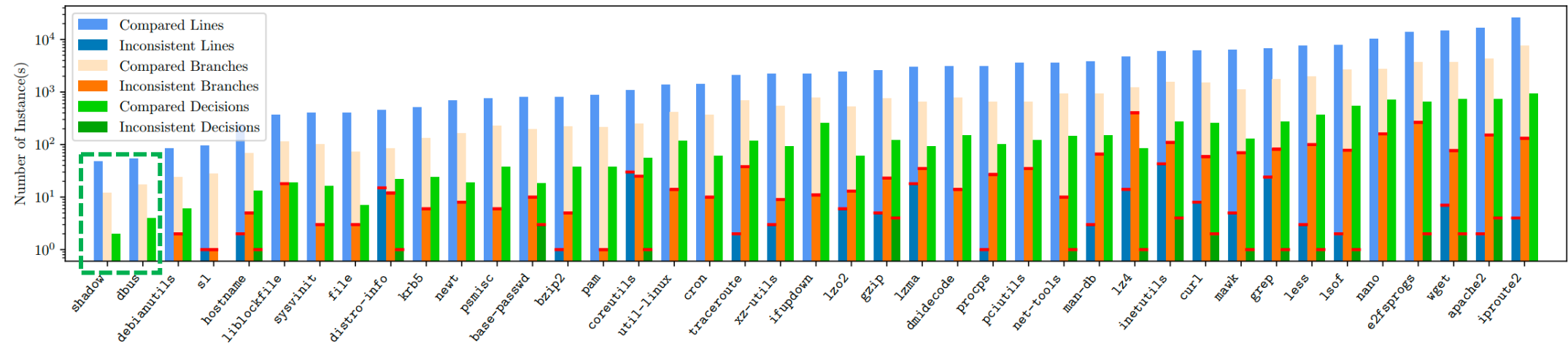
Inconsistencies for Simple Commands



	Lines	Branches	MC/DC
Total compared	168,549	44,126	7,527
Total inconsistent (number + value)	199	2,098 (1,884 + 214)	30 (29 + 1)

- **39** packages exhibit in total **2,327** inconsistencies
- Sampled **1,240** for inspection

Inconsistencies for Simple Commands



	Lines	Branches	MC/DC
Total compared	168,549	44,126	7,527
Total inconsistent		2,098	30
(number + value)	199	(1,884 + 214)	(29 + 1)

- **39** packages exhibit in total **2,327** inconsistencies
- Sampled **1,240** for inspection
 1. Bugs (256)
 2. Conventions (954)
 3. Compiler-specific differences (30)

Bugs Found by This Work

- **2** crashing bugs in LLVM-cov

Bugs Found by This Work

- **2** crashing bugs in LLVM-cov
- **32** wrong coverage bugs

#	Bug ID	Tool	Incons. Type	Affected Package(s)	Occur.	Triggering Condition(s)
1	120321	Gcov	L	9 (lzo2)	6 + 11	constant, loop, no code, variable scope
2	120478	Gcov	L	2 (lzo2)	3 + 2	inline function, multistmt
3	117412	Gcov	L	3 (lz4)	3 + 1	dereference, function, multiline, type conversion
4	120482	Gcov	L	1 (lz4)	3 + n/a	const keyword, inline function
5	120490	Gcov	L	1 (lz4)	3 + n/a	inline function, switch-case
6	117415	Gcov	L	1 (apache2)	1 + n/a	dereference, function, if, loop, multiline
7	120348	Gcov	L	1 (lzma)	1 + n/a	C++, extern "C", function, loop, return, variable scope
8	120484	Gcov	L	1 (curl)	1 + n/a	if, loop, variable scope
9	120489	Gcov	L	1 (lz4)	1 + n/a	continue, dereference, loop, variable scope
10	120491	Gcov	L	1 (lzma)	1 + n/a	C++, constructor, multiline
11	120492	Gcov	L	1 (lzma)	1 + n/a	C++ standard library, type conversion
12	120486	Gcov	L/BN	5 (distro-info)	6 + 19	multiline, ternary
13	120319	Gcov	L/BN	1 (lzma)	3 + n/a	C++, function, multiline, variable scope
14	120332	Gcov	L/BN/BV	5 (lzo2)	4 + 16	if, loop, no code, switch-case
15	120485	Gcov	BV	1 (mawk)	23 + 0	preprocessor directive, header
16	140427	LLVM-cov	L	3 (mawk)	2 + 2	goto, if, loop, macro, return
17	114622	LLVM-cov	L	1 (sl)	1 + n/a	break, header, macro
18	116884	LLVM-cov	L	1 (grep)	1 + n/a	break, constant, loop
19	105341	LLVM-cov	L/BV	1 (hostname)	1 + n/a	concurrency
20	116902	LLVM-cov	BN	1 (lzma)	3 + n/a	C++ standard library, function
21	101241	LLVM-cov	BN/MN	1 (net-tools)	2 + n/a	comma, constant, tautological compare, type conversion
22	131505	LLVM-cov	BN/BV/MN	1 (hostname)	1 + n/a	dereference, header, macro, type conversion
23	121914	Gcov	L	1 (lzo2)	0 + 4	inline function
24	121897	Gcov	L	2 (ifupdown)	0 + 3	function, variable scope
25	121901	Gcov	L	1 (bzip2)	0 + 3	dereference, loop, malloc, multiline
26	121896	Gcov	L	1 (mawk)	0 + 2	function, if, pointer
27	121902	Gcov	L	1 (ifupdown)	0 + 1	continue
28	158003	LLVM-cov	L	1 (mawk)	0 + 4	macro, switch-case
29	158080	LLVM-cov	L	2 (distro-info, mawk)	0 + 4	break, if, switch-case
30	157959	LLVM-cov	L	2 (lzo2, procp)	0 + 2	goto, if
31	157946	LLVM-cov	L	1 (distro-info)	0 + 1	preprocessor directive, header, macro
32	157981	LLVM-cov	L	1 (psmisc)	0 + 1	loop, multiline

Bugs Found by This Work

- **2** crashing bugs in LLVM-cov
- **32** wrong coverage bugs

#	Bug ID	Tool	Incons. Type	Affected Package(s)	Occur.	Triggering Condition(s)
1	120321	Gcov	L	9 (lzo2)	6 + 11	constant, loop, no code, variable scope
2	120478	Gcov	L	2 (lzo2)	3 + 2	inline function, multistmt
3	117412	Gcov	L	3 (lz4)	3 + 1	dereference, function, multiline, type conversion
4	120482	Gcov	L	1 (lz4)	3 + n/a	const keyword, inline function
5	120490	Gcov	L	1 (lz4)	3 + n/a	inline function, switch-case
6	117415	Gcov	L	1 (apache2)	1 + n/a	dereference, function, if, loop, multiline
7	120348	Gcov	L	1 (lzma)	1 + n/a	C++, extern "C", function, loop, return, variable scope
8	120484	Gcov	L	1 (curl)	1 + n/a	if, loop, variable scope
9	120489	Gcov	L	1 (lz4)	1 + n/a	continue, dereference, loop, variable scope
10	120491	Gcov	L	1 (lzma)	1 + n/a	C++, constructor, multiline
11	120492	Gcov	L	1 (lzma)	1 + n/a	C++ standard library, type conversion
12	120486	Gcov	L/BN	5 (distro-info)	6 + 19	multiline, ternary
13	120319	Gcov	L/BN	1 (lzma)	3 + n/a	C++, function, multiline, variable scope
14	120332	Gcov	L/BN/BV	5 (lzo2)	4 + 16	if, loop, no code, switch-case
15	120485	Gcov	BV	1 (mawk)	23 + 0	preprocessor directive, header
16	140427	LLVM-cov	L	3 (mawk)	2 + 2	goto, if, loop, macro, return
17	114622	LLVM-cov	L	1 (sl)	1 + n/a	break, header, macro
18	116884	LLVM-cov	L	1 (grep)	1 + n/a	break, constant, loop
19	105341	LLVM-cov	L/BV	1 (hostname)	1 + n/a	concurrency
20	116902	LLVM-cov	BN	1 (lzma)	3 + n/a	C++ standard library, function
21	101241	LLVM-cov	BN/MN	1 (net-tools)	2 + n/a	comma, constant, tautological compare, type conversion
22	131505	LLVM-cov	BN/BV/MN	1 (hostname)	1 + n/a	dereference, header, macro, type conversion
23	121914	Gcov	L	1 (lzo2)	0 + 4	inline function
24	121897	Gcov	L	2 (ifupdown)	0 + 3	function, variable scope
25	121901	Gcov	L	1 (bzip2)	0 + 3	dereference, loop, malloc, multiline
26	121896	Gcov	L	1 (mawk)	0 + 2	function, if, pointer
27	121902	Gcov	L	1 (ifupdown)	0 + 1	continue
28	158003	LLVM-cov	L	1 (mawk)	0 + 4	macro, switch-case
29	158080	LLVM-cov	L	2 (distro-info, mawk)	0 + 4	break, if, switch-case
30	157959	LLVM-cov	L	2 (lzo2, procp)	0 + 2	goto, if
31	157946	LLVM-cov	L	1 (distro-info)	0 + 1	preprocessor directive, header, macro
32	157981	LLVM-cov	L	1 (psmisc)	0 + 1	loop, multiline

Bugs Found by This Work

- **2** crashing bugs in LLVM-cov
- **32** wrong coverage bugs

• In both tools: **20** in Gcov and **12** in LLVM-cov

#	Bug ID	Tool	Incons. Type	Affected Package(s)	Occur.	Triggering Condition(s)
1	120321	Gcov	L	1 (lzo2)	6 + 11	constant, loop, no code, variable scope
2	120478	Gcov	L	1 (lzo2)	3 + 2	constant, loop, no code, variable scope
3	117412	Gcov	L	3 (lz4)	3 + 1	dereference, function, multiline, type conversion
4	120482	Gcov	L	1 (lz4)	3 + n/a	const keyword, inline function
5	120490	Gcov	L	1 (lz4)	3 + n/a	inline function, switch-case
6	117415	Gcov	L	1 (apache2)	1 + n/a	dereference, function, if, loop, multiline
7	120348	Gcov	L	1 (lzma)	1 + n/a	C++, extern "C", function, loop, return, variable scope
8	120484	Gcov	L	1 (curl)	1 + n/a	if, loop, variable scope
9	120489	Gcov	L	1 (lz4)	1 + n/a	continue, dereference, loop, variable scope
10	120491	Gcov	L	1 (lzma)	1 + n/a	C++, constructor, multiline
11	120492	Gcov	L	1 (lzma)	1 + n/a	C++ standard library, type conversion
12	120486	Gcov	L/BN	5 (distro-info)	6 + 19	multiline, ternary
13	120319	Gcov	L/BN	1 (lzma)	3 + n/a	C++, function, multiline, variable scope
14	120332	Gcov	L/BN/BV	5 (lzo2)	4 + 16	if, loop, no code, switch-case
15	120485	Gcov	BV	1 (mawk)	23 + 0	preprocessor directive, header
16	140427	LLVM-cov	L	3 (mawk)	2 + 2	goto, if, loop, macro, return
17	114622	LLVM-cov	L	1 (sl)	1 + n/a	break, header, macro
18	116884	LLVM-cov	L	1 (grep)	1 + n/a	break, constant, loop
19	105341	LLVM-cov	L/BV	1 (hostname)	1 + n/a	concurrency
20	116902	LLVM-cov	BN	1 (lzma)	3 + n/a	C++ standard library, function
21	101241	LLVM-cov	BN/MN	1 (net-tools)	2 + n/a	comma, constant, tautological compare, type conversion
22	131505	LLVM-cov	BN/BV/MN	1 (hostname)	1 + n/a	dereference, header, macro, type conversion
23	121914	Gcov	L	1 (lzo2)	0 + 4	inline function
24	121897	Gcov	L	2 (ifupdown)	0 + 3	function, variable scope
25	121901	Gcov	L	1 (bzip2)	0 + 3	dereference, loop, malloc, multiline
26	121896	Gcov	L	1 (mawk)	0 + 2	function, if, pointer
27	121902	Gcov	L	1 (ifupdown)	0 + 1	continue
28	158003	LLVM-cov	L	1 (mawk)	0 + 4	macro, switch-case
29	158080	LLVM-cov	L	2 (distro-info, mawk)	0 + 4	break, if, switch-case
30	157959	LLVM-cov	L	2 (lzo2, procps)	0 + 2	goto, if
31	157946	LLVM-cov	L	1 (distro-info)	0 + 1	preprocessor directive, header, macro
32	157981	LLVM-cov	L	1 (psmisc)	0 + 1	loop, multiline

Bugs Found by This Work

- **2** crashing bugs in LLVM-cov
- **32** wrong coverage bugs

#	Bug ID	Tool	Incons. Type	Affected Package(s)	Occur.	Triggering Condition(s)
1	120321	Gcov	L	1 (lzo2)	6 + 11	constant, loop, no code, variable scope
2	120478	Gcov	L	1 (lzo2)	3 + 2	constant, loop, no code, variable scope
3	117412	Gcov	L	3 (lzo4)	3 + 1	dereference, function, multiline, type conversion
4	120482	Gcov	L	1 (lzo4)	3 + n/a	const keyword, inline function
5	120490	Gcov	L	1 (lzo4)	3 + n/a	inline function, switch-case
6	117415	Gcov	L	1 (lzo4)	3 + n/a	inline function, switch-case
7	120348	Gcov	L	1 (lzma)	1 + n/a	C++, extern "C", function, loop, return, variable scope
8	120484	Gcov	L	1 (curl)	1 + n/a	if, loop, variable scope
9	120489	Gcov	L	1 (lzo4)	1 + n/a	continue, dereference, loop, variable scope
10	120491	Gcov	L	1 (lzma)	1 + n/a	C++, constructor, multiline
11	120492	Gcov	L	1 (lzma)	1 + n/a	C++ standard library, type conversion
12	120486	Gcov	L/BN	5 (distro-info)	6 + 19	multiline, ternary
13	120319	Gcov	L/BN	1 (lzma)	3 + n/a	C++, function, multiline, variable scope
14	120332	Gcov	L/BN/BV	5 (lzo2)	4 + 16	if, loop, no code, switch-case
15	120485	Gcov	BV	1 (mawk)	23 + 0	preprocessor directive, header
16	140427	LLVM-cov	L	3 (mawk)	2 + 2	goto, if, loop, macro, return
17	114622	LLVM-cov	L	1 (sl)	1 + n/a	break, header, macro
18	116884	LLVM-cov	L	1 (grep)	1 + n/a	break, constant, loop
19	105341	LLVM-cov	L/BV	1 (hostname)	1 + n/a	concurrency
20	116902	LLVM-cov	BN	1 (lzma)	3 + n/a	C++ standard library, function
21	101241	LLVM-cov	BN/MN	1 (net-tools)	2 + n/a	comma, constant, tautological compare, type conversion
22	131505	LLVM-cov	BN/BV/MN	1 (hostname)	1 + n/a	dereference, header, macro, type conversion
23	121914	Gcov	L	1 (lzo2)	0 + 4	inline function
24	121897	Gcov	L	2 (ifupdown)	0 + 3	function, variable scope
25	121901	Gcov	L	1 (bzip2)	0 + 3	dereference, loop, malloc, multiline
26	121896	Gcov	L	1 (mawk)	0 + 2	function, if, pointer
27	121902	Gcov	L	1 (ifupdown)	0 + 1	continue
28	158003	LLVM-cov	L	1 (mawk)	0 + 4	macro, switch-case
29	158080	LLVM-cov	L	2 (distro-info, mawk)	0 + 4	break, if, switch-case
30	157959	LLVM-cov	L	2 (lzo2, procps)	0 + 2	goto, if
31	157946	LLVM-cov	L	1 (distro-info)	0 + 1	preprocessor directive, header, macro
32	157981	LLVM-cov	L	1 (psmisc)	0 + 1	loop, multiline

• In both tools: **20** in Gcov and **12** in LLVM-cov

• Across all 3 metrics; one bug can affect multiple

Bugs Found by This Work

- **2** crashing bugs in LLVM-cov
- **32** wrong coverage bugs

#	Bug ID	Tool	Incons. Type	Affected Package(s)	Occur.	Triggering Condition(s)
1	120321	Gcov	L	5 (lzo2)	6 + 11	constant, loop, no code, variable scope
2	120478	Gcov	L	3 (lzo2)	3 + 2	constant, loop, no code, variable scope
3	117412	Gcov	L	3 (lzo2)	3 + 1	dereference, function, multiline, type conversion
4	120482	Gcov	L	1 (lzo2)	3 + n/a	const keyword, inline function
5	120490	Gcov	L	1 (lzo2)	3 + n/a	inline function, switch-case
6	117415	Gcov	L	1 (lzo2)	3 + n/a	inline function, switch-case
7	120348	Gcov	L	1 (lzo2)	1 + n/a	C++, extern "C", function, loop, return, variable scope
8	120484	Gcov	L	1 (curl)	1 + n/a	if, loop, variable scope
9	120489	Gcov	L	1 (lzo2)	1 + n/a	continue, dereference, loop, variable scope
10	120491	Gcov	L	1 (lzo2)	1 + n/a	C++, constructor, multiline
11	120492	Gcov	L	1 (lzo2)	1 + n/a	C++ standard library, type conversion
12	120486	Gcov	L/BN	5 (distro-info)	6 + 19	multiline, ternary
13	120319	Gcov	L/BN	1 (lzo2)	3 + n/a	C++, function, multiline, variable scope
14	120332	Gcov	L/BN/BV	5 (lzo2)	4 + 16	if, loop, no code, switch-case
15	120485	Gcov	BV	1 (mawk)	23 + 0	preprocessor directive, header
16	140427	LLVM-cov	L	3 (mawk)	2 + 2	goto, if, loop, macro, return
17	114622	LLVM-cov	L	1 (sl)	1 + n/a	break, header, macro
18	116884	LLVM-cov	L	1 (grep)	1 + n/a	break, constant, loop
19	105341	LLVM-cov	L/BV	1 (hostname)	1 + n/a	concurrency
20	116902	LLVM-cov	BN	1 (lzo2)	3 + n/a	C++ standard library, function
21	101241	LLVM-cov	BN/MN	1 (net-tools)	2 + n/a	comma, constant, tautological compare, type conversion
22	131505	LLVM-cov	BN/BV/MN	1 (hostname)	1 + n/a	dereference, header, macro, type conversion
23	121914	Gcov	L	1 (lzo2)	0 + 4	inline function
24	121897	Gcov	L	2 (ifupdown)	0 + 3	function, variable scope
25	121901	Gcov	L	1 (bzip2)	0 + 3	dereference, loop, malloc, multiline
26	121896	Gcov	L	1 (mawk)	0 + 2	function, if, pointer
27	121902	Gcov	L	1 (ifupdown)	0 + 1	continue
28	158003	LLVM-cov	L	1 (mawk)	0 + 4	macro, switch-case
29	158080	LLVM-cov	L	2 (distro-info, mawk)	0 + 4	break, if, switch-case
30	157959	LLVM-cov	L	2 (lzo2, procp)	0 + 2	goto, if
31	157946	LLVM-cov	L	1 (distro-info)	0 + 1	preprocessor directive, header, macro
32	157981	LLVM-cov	L	1 (psmisc)	0 + 1	loop, multiline

- In both tools: **20** in Gcov and **12** in LLVM-cov
- Across all 3 metrics; one bug can affect multiple
- Recurring patterns

Bugs Found by This Work

- **2** crashing bugs in LLVM-cov
- **32** wrong coverage bugs

#	Bug ID	Tool	Incons. Type	Affected Package(s)	Occur.	Triggering Condition(s)
1	120321	Gcov	L	1 (lzo2)	6 + 11	constant, loop, no code, variable scope
2	120478	Gcov	L	1 (lzo2)	3 + 2	constant, loop, no code, variable scope
3	117412	Gcov	L	3 (lzo4)	3 + 1	dereference, function, multiline, type conversion
4	120482	Gcov	L	1 (lzo4)	3 + n/a	const keyword, inline function
5	120490	Gcov	L	1 (lzo4)	3 + n/a	inline function, switch-case
6	117415	Gcov	L	1 (lzo4)	3 + n/a	inline function, switch-case
7	120348	Gcov	L	1 (lzma)	1 + n/a	C++, extern "C", function, loop, return, variable scope
8	120484	Gcov	L	1 (curl)	1 + n/a	if, loop, variable scope
9	120489	Gcov	L	1 (lzo4)	1 + n/a	continue, dereference, loop, variable scope
10	120491	Gcov	L	1 (lzma)	1 + n/a	C++, constructor, multiline
11	120492	Gcov	L	1 (lzma)	1 + n/a	C++ standard library, type conversion
12	120486	Gcov	L/BN	5 (distro-info)	6 + 19	multiline, ternary
13	120319	Gcov	L	1 (mawk)	23 + 0	C++, function, multiline, variable scope
14	120332	Gcov	L/BN/BV	3 (lzo4)	4 + 1	if, loop, no code, switch-case
15	120485	Gcov	BV	1 (mawk)	23 + 0	preprocessor directive, header
16	140427	LLVM-cov	L	2 (mawk)	2 + 2	goto, if, loop, macro, return
17	114622	LLVM-cov	L	1 (mawk)	1 + n/a	break, header, macro
18	116884	LLVM-cov	L	1 (grep)	1 + n/a	break, constant, loop
19	105341	LLVM-cov	L	1 (hostname)	1 + n/a	concurrency
20	116902	LLVM-cov	L	1 (lzma)	3 + n/a	C++ standard library, function
21	101241	LLVM-cov	BN/MN	1 (net-tools)	2 + n/a	comma, constant, tautological compare, type conversion
22	131505	LLVM-cov	BN/MN	1 (net-tools)	1 + n/a	dereference, header, macro, type conversion
23	121914	Gcov	L	1 (lzo2)	0 + 4	inline function
24	121897	Gcov	L	2 (ifupdown)	0 + 3	function, variable scope
25	121901	Gcov	L	1 (lzo2)	0 + 3	dereference, loop, malloc, multiline
26	121896	Gcov	L	1 (lzo2)	0 + 2	function, if, pointer
27	121902	Gcov	L	1 (ifupdown)	0 + 1	continue
28	158003	LLVM-cov	L	1 (mawk)	0 + 4	macro, switch-case
29	158080	LLVM-cov	L	2 (distro-info, mawk)	0 + 4	break, if, switch-case
30	157959	LLVM-cov	L	2 (lzo2, procps)	0 + 2	goto, if
31	157946	LLVM-cov	L	1 (distro-info)	0 + 1	preprocessor directive, header, macro
32	157981	LLVM-cov	L	1 (psmisc)	0 + 1	loop, multiline

- In both tools: **20** in Gcov and **12** in LLVM-cov
- Across all 3 metrics; one bug can affect multiple
- Recurring patterns
 - Coverage mapping
 - Coding style
 - C++
 - Constants
 - “No code”

Bugs Found by This Work

- **2** crashing bugs in LLVM-cov
- **32** wrong coverage bugs

#	Bug ID	Tool	Incons. Type	Affected Package(s)	Occur.	Triggering Condition(s)
1	120321	Gcov	L	1 (lzo2)	6 + 11	constant, loop, no code, variable scope
2	120478	Gcov	L	1 (lzo2)	3 + 2	constant, loop, no code, variable scope
3	117412	Gcov	L	3 (lzo4)	3 + 1	dereference, function, multiline, type conversion
4	120482	Gcov	L	1 (lzo4)	3 + n/a	const keyword, inline function
5	120490	Gcov	L	1 (lzo4)	3 + n/a	inline function, switch-case
6	117415	Gcov	L	1 (lzo4)	3 + n/a	inline function, switch-case
7	120348	Gcov	L	1 (lзма)	1 + n/a	C++, extern "C", function, loop, return, variable scope
8	120484	Gcov	L	1 (curl)	1 + n/a	if, loop, variable scope
9	120489	Gcov	L	1 (lzo4)	1 + n/a	continue, dereference, loop, variable scope
10	120491	Gcov	L	1 (lзма)	1 + n/a	C++, constructor, multiline
11	120492	Gcov	L	1 (lзма)	1 + n/a	C++ standard library, type conversion
12	120486	Gcov	L/BN	5 (distro-info)	6 + 19	multiline, ternary
13	120319	Gcov	L	1 (mawk)	23 + 0	C++, function, multiline, variable scope
14	120332	Gcov	L/BN	3 (lzo4)	4 + 1	if, loop, no code, switch-case
15	120485	Gcov	BV	1 (mawk)	23 + 0	preprocessor directive, header
16	140427	LLVM-cov	L	2 (mawk)	2 + 2	goto, if, loop, macro, return
17	114622	LLVM-cov	L	1 (mawk)	1 + n/a	break, header, macro
18	116884	LLVM-cov	L	1 (grep)	1 + n/a	break, constant, loop
19	105341	LLVM-cov	L	1 (hostname)	1 + n/a	concurrency
20	116902	LLVM-cov	L	1 (lзма)	3 + n/a	C++ standard library, function
21	101241	LLVM-cov	BN/MN	1 (net-tools)	2 + n/a	comma, constant, tautological compare, type conversion
22	131505	LLVM-cov	BN/MN	1 (net-tools)	1 + n/a	dereference, header, macro, type conversion
23	121914	Gcov	L	1 (mawk)	0 + 4	inline function
24	121897	Gcov	L	2 (ifupdown)	0 + 3	function, variable scope
25	121901	Gcov	L	1 (mawk)	0 + 3	dereference, loop, malloc, multiline
26	121896	Gcov	L	1 (mawk)	0 + 2	function, if, pointer
27	121902	Gcov	L	1 (ifupdown)	0 + 1	continue
28	158003	LLVM-cov	L	1 (mawk)	0 + 4	macro, switch-case
29	158080	LLVM-cov	L	2 (distro-info, mawk)	0 + 4	break, if, switch-case
30	157959	LLVM-cov	L	2 (lzo2, procps)	0 + 2	goto, if
31	157946	LLVM-cov	L	1 (distro-info)	0 + 1	preprocessor directive, header, macro
32	157981	LLVM-cov	L	1 (psmisc)	0 + 1	loop, multiline

- In both tools: **20** in Gcov and **12** in LLVM-cov
- Across all 3 metrics; one bug can affect multiple
- Recurring patterns
 - Coverage mapping
 - Coding style
 - C++
 - Constants
 - “No code”

Summary

- Real-world projects help reveal unique coverage tool bugs
- DEBCOVDIFF: a differential testing framework
- Revealed 34 new bugs in Gcov and LLVM-cov
- Open sourced at <https://github.com/xlab-uiuc/DEBCOVDIFF>



Future Directions

- More challenging input than Debian packages, such as Linux kernel or Xen hypervisor
- Aspirational goal: qualify open-source coverage measurement tools, e.g., Gcov or LLVM-cov, per DO-330 for certifying software to DO-178C